TESTNG CLASS 6

ITestListener and ExtentReports

Beginner-friendly notes with annotated Java code, Selenium scenarios, screenshots, HTML reporting, XML setup, troubleshooting, and industry best practices.

Learning objective
By the end of this guide, you should be able to explain TestNG listener callbacks, register a listener in testng.xml, create an Extent HTML report, log PASS/FAIL/SKIP outcomes, and attach Selenium screenshots to test results.

Based on the uploaded Class 6 lecture notes, with practical clarifications and corrected examples.

How to use this guide

1. Read the listener lifecycle and understand when each callback fires.
2. Study the corrected ExtentReports listener and the supporting screenshot/driver utilities.
3. Run the example from testng.xml, inspect the HTML report, and deliberately fail one assertion to verify the failure screenshot.
4. Complete the practice tasks and use the troubleshooting section when the report or screenshots are missing.

Contents

- 1. What is a TestNG listener?
- 2. ITestListener lifecycle and callbacks
- 3. ITestResult and ITestContext explained
- 4. Registering a listener in testng.xml
- 5. ExtentReports: dependency and core concepts
- 6. Important corrections to the lecture example
- 7. A maintainable implementation: driver context and screenshot utility
- 8. Complete ExtentTestListener implementation
- 9. SauceDemo Selenium example
- 10. Execution flow and expected report output
- 11. Real-world scenarios and extensions
- 12. Best practices and common mistakes
- 13. Troubleshooting checklist
- 14. Hands-on exercises with suggested answers
- 15. Interview questions and quick revision

Core idea
A listener observes test execution events and reacts to them. The test method should still contain the actual test actions and assertions; the listener should handle cross-cutting concerns such as reporting, logging, and diagnostic screenshots.

1. What is a TestNG listener?

A listener is a class that implements a TestNG listener interface. TestNG calls its callback methods automatically when specified events happen during execution. You do not manually call onTestStart() or onTestFailure() from your test methods.

Think of it as an event handler. A web application can raise an event when a user clicks a button; similarly, TestNG raises lifecycle events when a test starts, passes, fails, or is skipped.

Why listeners are useful

- Centralize reporting so every test does not need repeated report-writing code.
- Capture screenshots and diagnostic information when a Selenium test fails.
- Record test names, statuses, exception messages, timing, and group names.
- Integrate with reporting tools such as ExtentReports; other tools can use their own adapters or listeners.
- Keep test methods focused on application behavior and assertions.

Typical listener workflow

1. TestNG starts the configured suite/test execution.
2. The listener receives lifecycle callbacks.
3. At test start, create a report entry for the method.
4. At completion, log PASS, FAIL, or SKIP, and attach a screenshot when appropriate.
5. At suite end, flush the report so the HTML file is written completely.

Important distinction
ITestListener is for test-method events. Its onStart(ITestContext) and onFinish(ITestContext) methods describe a TestNG <test> context, not the entire suite. Use ISuiteListener.onStart(ISuite) and onFinish(ISuite) when you need a suite-level start and finish hook.

2. ITestListener lifecycle and callbacks

Callback	When it runs	Typical action
onStart(ITestContext)	Before test methods under a given XML <test> context begin.	Log the context name or initialize context-specific data.
onTestStart(ITestResult)	When an individual @Test method is about to run.	Create an ExtentTest entry, log method name and groups.
onTestSuccess(ITestResult)	After a test method succeeds.	Log PASS; optionally attach a success screenshot.
onTestFailure(ITestResult)	After a test method fails.	Log FAIL and the error; capture screenshot and diagnostics.
onTestSkipped(ITestResult)	When a test method is skipped.	Log SKIP and the skip reason if available.
onFinish(ITestContext)	After test methods in that XML <test> context complete.	Log context summary; do not assume it is suite finish.

TestNG has additional callback methods in listener interfaces, but these six are the main ITestListener methods used in the supplied lecture.

Suite-level callbacks

Interface/method	Meaning
ISuiteListener.onStart(ISuite)	Runs when the whole suite starts. Good place to initialize a suite-wide HTML report.
ISuiteListener.onFinish(ISuite)	Runs when the whole suite finishes. Good place to flush the report once.
IExecutionListener	Can observe the beginning and end of the overall TestNG execution, which may include suites.

Why the distinction matters

An XML suite can contain several <test> blocks (for example, smoke and regression). If you create and flush the same report inside ITestListener.onStart/onFinish, it may initialize or flush once per <test> context. For a single report across the suite, suite-level lifecycle is clearer.

3. ITestResult and ITestContext explained

ITestResult

ITestResult describes one test-method execution. It lets the listener inspect the method name, its status, duration, groups, and any throwable that caused failure or skip.

```
String methodName = result.getMethod().getMethodName();
Throwable error = result.getThrowable();
long durationMillis = result.getEndMillis() - result.getStartMillis();
```

Expression	What it provides
result.getMethod().getMethodName()	The Java test method name, e.g. login_with_valid_credentials.
result.getThrowable()	The exception/error recorded for the test, or null when no throwable is recorded.
result.getStatus()	TestNG status value (for example, success, failure, or skip).
result.getStartMillis()/getEndMillis()	Timestamps that can be used to calculate elapsed duration.

ITestContext

ITestContext represents a TestNG <test> context from testng.xml. It gives access to the context name, suite, included/excluded test methods, and the current result counts.

```
String contextName = context.getName();
String suiteName = context.getSuite().getName();
int passed = context.getPassedTests().size();
int failed = context.getFailedTests().size();
int skipped = context.getSkippedTests().size();
```

Beginner tip
The name <test> in testng.xml does not mean one @Test method. It is a configuration block that may contain many Java classes and many @Test methods.

4. Registering a listener in testng.xml

A listener must be registered for TestNG to discover and call it. The lecture uses XML registration, which is a common approach for applying a listener to all classes in a suite.

File: testng.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="Sauce Demo Application">
  <listeners>
    <listener class-name="tutorial1.ExtentTestListener"/>
  </listeners>

  <test name="Regression Test Cases">
    <classes>
      <class name="tutorial1.SauceDemoTest"/>
    </classes>
  </test>
</suite>
```

Line-by-line explanation

- <suite name="..."> gives the suite a readable name.
- <listeners> contains listener registrations.
- class-name must match the fully qualified Java package and class name exactly.
- <test> defines a TestNG XML context; its classes are selected for execution.
- <class name="tutorial1.SauceDemoTest"/> tells TestNG which Java test class to run.

Common XML mistakes

- Using <listeners> as the closing tag instead of </listeners>.
- Misspelling the package name or listener class name.
- Placing listener configuration outside the suite root.
- Editing testng.xml but still running the Java class directly in an IDE configuration that does not use that XML.

5. ExtentReports: dependency and core concepts

ExtentReports is a reporting library that can generate an HTML report containing test names, statuses, logs, and attachments such as screenshots. The supplied lecture uses ExtentReports 5.0.9 with ExtentSparkReporter.

Maven dependency from the lecture

```
<dependency>
  <groupId>com.aventstack</groupId>
  <artifactId>extentreports</artifactId>
  <version>5.0.9</version>
</dependency>
```

Add the dependency inside the <dependencies> element in pom.xml. Use a version approved for your project and verify compatibility with your Java/build setup before changing a dependency version.

Main classes

Class	Purpose
ExtentReports	Report engine that manages report entries and writes results when flushed.
ExtentSparkReporter	Creates the HTML (Spark) report output at a specified file location.
ExtentTest	One test entry in the report. Use it to log PASS, FAIL, SKIP, messages, and attachments.
Status	Enumeration describing a report log status, commonly PASS, FAIL, and SKIP.
addScreenCaptureFromPath(path)	Attaches a screenshot file path to the report entry.

Report lifecycle

```
ExtentReports extent = new ExtentReports();
ExtentSparkReporter spark = new ExtentSparkReporter("reports/ExtentReport.html");
extent.attachReporter(spark);
ExtentTest test = extent.createTest("login_with_valid_credentials");
test.log(Status.PASS, "Login page opened successfully");
```

```
extent.flush();
```

The report entry is created by createTest(), the result is logged through log(), and flush() writes outstanding report data to disk. In the complete example, the listener owns this work so individual tests do not repeat it.

Do not forget flush()
If you never call flush(), the HTML report may be missing, incomplete, or not reflect the latest test results. Flush at the appropriate lifecycle boundary; do not flush after every assertion or log entry.

6. Important corrections to the lecture example

Lecture pattern / issue	Recommended correction and reason
Treats ITestListener.onStart/onFinish as suite callbacks.	They refer to an ITestContext (<test> block). Use ISuiteListener for whole-suite report initialization and final flush.
Uses a single field: ExtentTest extentTest.	With parallel tests, another test can overwrite that field. Use ThreadLocal<ExtentTest> or a result-keyed concurrent map.
Logs a skipped test with Status.FAIL.	Use Status.SKIP. The report should match TestNG's actual outcome.
Creates screenshot paths without ensuring the folder exists.	Create directories with Files.createDirectories() before writing files.
Uses Windows-only path separators in strings.	Use java.nio.file.Path/Paths so paths work on Windows, macOS, and Linux.
Calls a static screenshot method on the test class.	Use a driver context or screenshot utility that safely handles an unavailable/closed driver.
Runs login, sort, and checkout as separate methods sharing one static driver.	This creates order and state coupling. Make tests independent or keep a multi-step end-to-end flow inside one @Test.
Catches IOException and only prints a stack trace.	Log the attachment failure in the report or console; do not let a secondary screenshot error hide the original test failure.

7. Maintainable implementation: driver context and screenshot utility

The following implementation separates responsibilities. DriverContext stores the current browser per thread, ScreenshotUtil creates a file, and the listener decides when to attach that file to the report. This is safer than using one static WebDriver field if parallel execution is later enabled.

7.1 DriverContext.java

```
package tutorial1;

import org.openqa.selenium.WebDriver;

public final class DriverContext {
   private static final ThreadLocal<WebDriver> DRIVER = new ThreadLocal<>();
```

```
    private DriverContext() { }

    public static void setDriver(WebDriver driver) {
        DRIVER.set(driver);
    }

    public static WebDriver getDriver() {
        return DRIVER.get();
    }

    public static void removeDriver() {
        DRIVER.remove();
    }
}
```

Explanation

- ThreadLocal stores a driver separately for each executing thread; this reduces accidental driver sharing during parallel tests.
- setDriver() is called immediately after browser creation.
- getDriver() is called by tests and by the listener when a screenshot is needed.
- removeDriver() clears the thread association after the browser is closed.

7.2 ScreenshotUtil.java

```
package tutorial1;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebDriver;

public final class ScreenshotUtil {
    private ScreenshotUtil() { }

    public static String capture(WebDriver driver, String testName, String status)
            throws IOException {
        if (driver == null) {
            return null;
        }

        String safeTestName = testName.replaceAll("[^a-zA-Z0-9._-]", "_");
        String timestamp = LocalDateTime.now().format(
                DateTimeFormatter.ofPattern("yyyyMMdd_HHmmss_SSS"));

        Path directory = Path.of(System.getProperty("user.dir"),
                "reports", "screenshots", status.toLowerCase());
        Files.createDirectories(directory);

        Path destination = directory.resolve(safeTestName + "_" + timestamp + ".png");
        Path source = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE).toPath();
        Files.copy(source, destination, StandardCopyOption.REPLACE_EXISTING);
        return destination.toAbsolutePath().toString();
    }
```

```
}
```

Explanation and safeguards

- The null check avoids a NullPointerException when browser setup failed or the driver is unavailable.
- The test name is sanitized so spaces and special characters do not create troublesome file names.
- A timestamp helps prevent one screenshot overwriting another.
- Files.createDirectories() creates the destination directory if it does not exist.
- The helper returns an absolute path so ExtentReports can resolve the attachment.

Screenshot timing
Capture a failure screenshot while the browser session is still available. The listener callback normally runs as the test outcome is reported; avoid closing the driver before failure diagnostics have been collected. If setup itself fails, driver may be null and a screenshot may not be possible.

8. Complete ExtentTestListener implementation

This listener implements both ITestListener and ISuiteListener. ITestListener handles individual test results, while ISuiteListener creates and flushes one report for the suite. ThreadLocal<ExtentTest> prevents the current report entry from being overwritten by another test running on a different thread.

File: ExtentTestListener.java

```
package tutorial1;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;

import org.testng.ISuite;
import org.testng.ISuiteListener;
import org.testng.ITestContext;
import org.testng.ITestListener;
import org.testng.ITestResult;

import com.aventstack.extentreports.ExtentReports;
import com.aventstack.extentreports.ExtentTest;
import com.aventstack.extentreports.Status;
import com.aventstack.extentreports.reporter.ExtentSparkReporter;

import org.openqa.selenium.WebDriver;

public class ExtentTestListener implements ITestListener, ISuiteListener {
   private ExtentReports extent;
   private final ThreadLocal<ExtentTest> currentTest = new ThreadLocal<>();

   @Override
   public void onStart(ISuite suite) {
      try {
         Path reportDirectory = Path.of(System.getProperty("user.dir"), "reports");
         Files.createDirectories(reportDirectory);
         Path reportFile = reportDirectory.resolve("ExtentReport.html");

         ExtentSparkReporter spark = new ExtentSparkReporter(reportFile.toString());
         extent = new ExtentReports();
         extent.attachReporter(spark);
         extent.setSystemInfo("Suite", suite.getName());
         extent.setSystemInfo("OS", System.getProperty("os.name"));
         extent.setSystemInfo("Java", System.getProperty("java.version"));
      } catch (IOException e) {
         throw new IllegalStateException("Could not initialize Extent report", e);
      }
```

```
}

@Override
public void onStart(ITestContext context) {
    System.out.println("Test context started: " + context.getName());
}

@Override
public void onTestStart(ITestResult result) {
    String name = result.getMethod().getMethodName();
    ExtentTest test = extent.createTest(name);
    currentTest.set(test);
    test.info("Test started: " + name);
}

@Override
public void onTestSuccess(ITestResult result) {
    logOutcome(result, Status.PASS, "Test passed", true);
}

@Override
public void onTestFailure(ITestResult result) {
    logOutcome(result, Status.FAIL, "Test failed", true);
}

@Override
public void onTestSkipped(ITestResult result) {
    logOutcome(result, Status.SKIP, "Test skipped", false);
}

private void logOutcome(ITestResult result, Status status, String message,
                        boolean takeScreenshot) {
    ExtentTest test = currentTest.get();
    if (test == null) {
        test = extent.createTest(result.getMethod().getMethodName());
        currentTest.set(test);
    }

    test.log(status, message + ": " + result.getMethod().getMethodName());
    if (result.getThrowable() != null) {
        test.log(status, result.getThrowable().toString());
    }

    if (takeScreenshot) {
        WebDriver driver = DriverContext.getDriver();
        try {
            String path = ScreenshotUtil.capture(driver,
                    result.getMethod().getMethodName(), status.name());
            if (path != null) {
                test.addScreenCaptureFromPath(path);
            } else {
                test.info("Screenshot not captured: WebDriver was unavailable.");
            }
        } catch (IOException | RuntimeException e) {
            test.warning("Screenshot attachment failed: " + e.getMessage());
        }
    }
    currentTest.remove();
}

@Override
public void onFinish(ITestContext context) {
    System.out.println("Test context finished: " + context.getName()
            + " | Passed=" + context.getPassedTests().size()
            + " | Failed=" + context.getFailedTests().size()
            + " | Skipped=" + context.getSkippedTests().size());
}

@Override
```

```
    public void onFinish(ISuite suite) {
        if (extent != null) {
            extent.flush();
        }
    }
}
```

How the listener works

1. onStart(ISuite) creates the reports folder, attaches ExtentSparkReporter, and records suite/system metadata.
2. onTestStart() creates a separate ExtentTest entry for the method and stores it in a ThreadLocal.
3. onTestSuccess(), onTestFailure(), and onTestSkipped() call one shared method with the correct status.
4. The shared method logs the outcome and throwable, then optionally captures and attaches a screenshot.
5. currentTest.remove() clears the current test entry from the thread after it is complete.
6. onFinish(ISuite) calls flush() once after the whole suite finishes.

Version note
ExtentReports API details can vary across major versions. This example follows the ExtentReports 5.x style used in the source notes. If your project uses another version, confirm the method signatures in that version before copying the code.

9. SauceDemo Selenium example

The source notes use SauceDemo to demonstrate login, sorting, adding an item, and checkout. The original sample creates the driver inside the login test and then relies on later test methods reusing the same static driver. That pattern is fragile because running a single method, retrying a method, or enabling parallel execution can break the shared state.

The improved sample creates a fresh browser before each test and includes login in each test that needs an authenticated session. For a true multi-step checkout journey, one end-to-end test can keep all dependent steps in the same method.

9.1 BaseTest.java

```
package tutorial1;

import java.time.Duration;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;

public class BaseTest {
    @BeforeMethod
    public void setUp() {
        WebDriver driver = new ChromeDriver();
        DriverContext.setDriver(driver);
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(0));
        driver.get("https://www.saucedemo.com/");
    }

    @AfterMethod(alwaysRun = true)
    public void tearDown() {
        WebDriver driver = DriverContext.getDriver();
        try {
```

```
            if (driver != null) {
                driver.quit();
            }
        } finally {
            DriverContext.removeDriver();
        }
    }
}
```

This example sets implicit wait to zero so that explicit waits can be used intentionally in larger test methods. Do not combine large implicit waits with explicit waits without understanding the resulting timing behavior.

9.2 SauceDemoTest.java

File: SauceDemoTest.java

```
package tutorial1;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.testng.Assert;
import org.testng.annotations.Test;
import java.time.Duration;

public class SauceDemoTest extends BaseTest {
    private void login(WebDriver driver) {
        driver.findElement(By.id("user-name")).sendKeys("standard_user");
        driver.findElement(By.id("password")).sendKeys("secret_sauce");
        driver.findElement(By.id("login-button")).click();
        new WebDriverWait(driver, Duration.ofSeconds(10)).until(
            ExpectedConditions.visibilityOfElementLocated(By.className("title")));
    }

    @Test
    public void login_with_valid_credentials() {
        WebDriver driver = DriverContext.getDriver();
        login(driver);
        Assert.assertTrue(driver.findElement(By.className("title")).isDisplayed(),
                "Products page should be displayed after login");
    }

    @Test
    public void checkout_bike_light_item() {
        WebDriver driver = DriverContext.getDriver();
        login(driver);
        driver.findElement(By.id("add-to-cart-sauce-labs-bike-light")).click();
        driver.findElement(By.className("shopping_cart_link")).click();
        Assert.assertTrue(driver.getPageSource().contains("Sauce Labs Bike Light"),
                "Cart should contain the selected item");
        driver.findElement(By.id("checkout")).click();
        driver.findElement(By.id("first-name")).sendKeys("Test");
        driver.findElement(By.id("last-name")).sendKeys("User");
        driver.findElement(By.id("postal-code")).sendKeys("560001");
        driver.findElement(By.id("continue")).click();
        Assert.assertTrue(driver.getCurrentUrl().contains("checkout-step-two"),
                "Checkout overview should be displayed");
    }
}
```

What to observe

- The listener is not called directly by the test class; TestNG invokes it because it was registered in testng.xml.

- The assertions decide whether the test passes or fails. ExtentReports only records the result; it does not make a failing test pass.
- If an assertion fails, the listener tries to capture the browser screenshot and attach it to the report.
- The BaseTest @AfterMethod closes the browser after each method; alwaysRun helps ensure cleanup even when a test fails.

Credential note
The SauceDemo credentials shown here are public demonstration credentials for the sample training application. For internal systems, never hard-code real passwords, access tokens, or customer information into test source code or HTML reports.

10. Execution flow and expected report output

Execution sequence

1. Run testng.xml from the IDE or Maven.
2. TestNG starts the suite and calls onStart(ISuite). ExtentReports is initialized.
3. TestNG starts the XML <test> context and calls onStart(ITestContext).
4. Before each @Test, TestNG calls the test listener onTestStart().
5. The test executes Selenium steps and assertions.
6. The listener logs PASS, FAIL, or SKIP and attaches a screenshot when configured.
7. After all tests and contexts finish, TestNG calls onFinish(ISuite); the listener flushes the report.

Report files

Path	Contents
reports/ExtentReport.html	HTML report with test method names, outcome, log entries, system information, and screenshot links.
reports/screenshots/pass/	Optional screenshots from successful tests.
reports/screenshots/fail/	Screenshots captured when tests fail.

The folder is created by the screenshot utility when needed. The HTML report is best viewed in a browser. In CI, archive the reports/ directory as a build artifact so the report remains available after the agent workspace is cleaned.

11. Real-world scenarios and extensions

Scenario A: Regression suite fails in CI

A login test fails only on the CI agent. The listener records the failed method, the exception string, and a browser screenshot. An engineer can open the report and inspect the page state without reproducing the failure immediately.

Scenario B: A test is skipped because a dependency failed

The listener should log SKIP, not FAIL. The skip reason is useful for separating a primary application defect from downstream tests that could not run.

Scenario C: Parallel browser tests

If multiple tests execute simultaneously, a shared ExtentTest variable can cause logs to appear under the wrong test. ThreadLocal<ExtentTest> keeps each thread associated with its own report entry, and ThreadLocal<WebDriver> keeps each thread associated with its own browser. The application and test setup must also be safe for parallel execution.

Scenario D: Report generation fails

Check whether the report directory is writable, whether the file is locked by an open process, whether ExtentReports is on the classpath, and whether the listener class name is correct. A screenshot failure should be logged as a diagnostic problem, not replace the original test failure.

Scenario E: Add a log for every major action

```
ExtentTest test = currentTest.get();
test.info("Opened login page");
test.info("Entered username");
test.info("Clicked Login");
test.pass("Products page was displayed");
```

In a reusable framework, avoid calling the listener's private fields directly from test code. If step-level reporting is needed, create a small reporting facade or page/action wrapper that obtains the current ExtentTest entry safely.

12. Best practices and common mistakes

Do this	Avoid this
Use ISuiteListener for suite-level report creation/flush.	Assume ITestListener onStart/onFinish mean the suite started/ended.
Use ThreadLocal or a concurrent result map for parallel reporting.	Store one shared mutable ExtentTest field for all parallel tests.
Log Status.SKIP for skipped tests.	Record skipped tests as failures.
Capture failure screenshots while the driver is alive.	Quit the browser before trying to take the screenshot.
Create output directories and use portable Path APIs.	Assume a folder exists or hard-code Windows-only separators.
Include useful exception details and readable test method names.	Log only "Test failed" with no reason.
Keep tests independent or group dependent steps into one journey.	Depend on method ordering and a driver created by a previous test.
Treat screenshot/report errors as secondary diagnostics.	Let a failed screenshot obscure the original assertion failure.
Keep secrets and personal data out of logs and screenshots.	Attach pages that expose tokens, passwords, or customer details without masking.
Archive HTML reports and screenshots in CI.	Assume the runner workspace will persist forever.

Screenshot strategy

- Failure-only screenshots are often enough for a normal suite and save disk space.
- Capturing every passing step can be useful during debugging but increases report size and execution cost.
- Ensure screenshots are unique and name them with test method, status, and timestamp.
- For headless runs, browser window dimensions should be set explicitly if screenshots need consistent dimensions.

Reporting security

- Do not log passwords, bearer tokens, private keys, session cookies, or sensitive customer fields.
- Restrict access to CI artifacts if screenshots may reveal internal data.
- Clean old reports based on a retention policy so screenshots do not accumulate indefinitely.

13. Troubleshooting checklist

Symptom	Likely cause	What to check
Listener methods never execute	Listener is not registered or class name is wrong.	Verify testng.xml uses <listeners> and the fully qualified class name; run that XML configuration.
HTML report not created	No flush, report initialization failed, or output folder is not writable.	Check console errors, suite callbacks, and reports path permissions.
Report exists but is empty	No test entries created or testng.xml selected no methods.	Check onTestStart events, class names, groups, and includes/excludes.
Test skipped shown as failed	Wrong Status constant in listener.	Use Status.SKIP in onTestSkipped().
Screenshot absent after failure	Driver unavailable, browser already quit, or directory/copy exception.	Check callback timing, DriverContext, screenshot folder creation, and logged warnings.
Screenshot attached but cannot open	Path is not accessible from report location or artifact was not copied.	Use absolute paths for local report, keep the screenshot beside archived report, and validate the file.
Logs appear under the wrong test in parallel mode	Shared mutable ExtentTest field.	Use ThreadLocal<ExtentTest> or a concurrent map keyed by ITestResult.
Report overwritten in multi-<test> XML suite	Report lifecycle managed per ITestContext instead of suite.	Create once in ISuiteListener.onStart() and flush in onFinish().
Listener screenshot error hides root error	Screenshot exception is not handled separately.	Catch screenshot I/O/runtime errors and retain the original test throwable.

14. Hands-on exercises with suggested answers

Exercise 1: Print the test method name on failure

Task: In onTestFailure(), get the method name from ITestResult and print it to the console.

Suggested answer

```
@Override
public void onTestFailure(ITestResult result) {
   System.out.println("FAILED: " + result.getMethod().getMethodName());
}
```

Exercise 2: Log skipped status correctly

Task: Fix this incorrect code: test.log(Status.FAIL, "Test skipped");

Suggested answer

```
test.log(Status.SKIP, "Test skipped: " + result.getMethod().getMethodName());
```

Exercise 3: Capture only on failures

Task: Modify the listener so it only calls ScreenshotUtil.capture() in onTestFailure(). Keep PASS and SKIP reporting without screenshots. This can reduce storage and HTML report size.

Exercise 4: Store browser and report paths portably

Task: Replace a hard-coded path like user.dir + "\\reports\\abc.html" with java.nio.file.Path.

Suggested answer

```
Path reportFile = Path.of(System.getProperty("user.dir"), "reports", "ExtentReport.html");
```

Exercise 5: Verify a report in a build pipeline

1. Run the suite with the intended testng.xml configuration.
2. Confirm reports/ExtentReport.html exists and opens in a browser.
3. Force one assertion to fail and check that the failure status, error message, and screenshot appear.
4. Archive reports/ExtentReport.html and reports/screenshots/ as CI artifacts.

15. Interview questions and quick revision

Q: What is a TestNG listener?

A: A callback mechanism that observes execution events and performs actions such as reporting, logging, or screenshots.

Q: What is ITestListener used for?

A: For per-test-method lifecycle events such as start, success, failure, and skip.

Q: What is the difference between ITestListener and ISuiteListener?

A: ITestListener observes test methods and a <test> context; ISuiteListener observes the entire suite lifecycle.

Q: What does ITestResult provide?

A: Information about one test execution, including method, status, throwable, and timing.

Q: Why is ExtentTest separate from ExtentReports?

A: ExtentReports manages the report; ExtentTest represents one method entry and its logs/status/attachments.

Q: Why do we call flush()?

A: To write pending report data to the HTML output.

Q: Why use ThreadLocal<ExtentTest>?

A: To prevent concurrent tests from sharing or overwriting the current report entry.

Q: Why should a skipped test use Status.SKIP?

A: The report status should match TestNG's test outcome and avoid misleading failure metrics.

Q: Why can screenshot capture fail?

A: The driver may be null/closed, the destination may not exist or be writable, or screenshot I/O can fail.

Q: Should screenshot failure change a passing application test into a failure?

A: Usually no. It should be recorded as a secondary diagnostic problem, while keeping the real test outcome clear.

Q: Where should the listener be registered?

A: It can be registered in testng.xml with a fully qualified class name, or through supported TestNG configuration mechanisms.

Q: What is the most important correction in the lecture example?

A: Use suite lifecycle callbacks for a suite-wide report, log SKIP correctly, and avoid shared state for parallel execution.

Quick revision sheet

Item	Remember
Listener	A listener observes lifecycle events; TestNG calls callbacks automatically.
Per-test callbacks	onTestStart, onTestSuccess, onTestFailure, onTestSkipped.
Context callbacks	onStart(ITestContext) and onFinish(ITestContext) concern an XML <test> context.
Suite callbacks	ISuiteListener.onStart/onFinish are suitable for a single suite-wide report lifecycle.
ExtentReports	ExtentReports manages output; ExtentSparkReporter creates HTML; ExtentTest logs one test.
Status	Use PASS, FAIL, SKIP accurately.
Screenshots	Capture while the browser is available; create directories; use unique portable paths.
Concurrency	Avoid shared WebDriver/ExtentTest fields when parallel execution is enabled.
CI	Archive the HTML report and screenshot folders as build artifacts.

Final takeaway

Build the listener once and keep it responsible for shared test infrastructure: results, logging, screenshots, and reports. Keep application actions and assertions inside the test code, use the correct callback lifecycle, and verify failure diagnostics in the HTML report before adopting the listener in CI.